

Department: CSE-(DS)	Academic Year: 2025-2026
Semester : 05	Group No. : TY-B5
Project Title: AI Agent for Code Vulnerability Detection	
Project Area: Cybersecurity, Agentic AI	

Group Members Details:

Sr. No.	Class & Div.	Roll No.	P.R. No.	Name of Student	Contact No.	Email ID
1	TY-B5	17	12414396	Shashank Kakad	9503291648	shashank.kakad24@vit.edu
2	TY-B5	34	12413287	Soham Moholkar	7276866053	soham.moholkar24@vit.edu
3	TY-B5	9	12411438	Omkar Kakade	9284874251	omkar.kakade24@vit.edu
4	TY-B5	45	12412189	Yash Pardeshi	8446791939	yash.pardeshi24@vit.edu

Name of Internal Guide: Dr. Nitin Sakhare Email ID: nitin.sakhare@vit.edu	Contact No: +91 8657258947
--	-----------------------------------

Project approved / Not approved		
Guide	Project Coordinator	Head of Department

Project Synopsis

1. Introduction

Modern software development has been transformed by AI-assisted coding tools such as GitHub Copilot, Cursor, and similar platforms. While these tools dramatically accelerate development velocity, they introduce a critical and underexamined problem: 48% of AI-generated code contains security vulnerabilities that traditional static analysis tools fail to detect. Existing solutions such as Snyk and Semgrep analyze code in isolation, entirely ignoring the relationship between application logic and the underlying database schema — the very relationship that governs the most severe classes of real-world vulnerabilities.

SentinelOps addresses this gap by deploying an AI agent that connects simultaneously to a company's source code repository and its database schema, builds a cross-layer knowledge graph, and uses Large Language Model reasoning to identify exploitable paths that no rule-based scanner can detect.

2. Problem Statement

Existing application security testing (AST) tools perform static analysis on code in isolation. They match patterns against known vulnerability signatures (e.g., SQL injection templates) but are fundamentally blind to the semantic relationship between code logic and the data it manipulates. Consider an API endpoint that writes user-controlled input to a `payment_status` column in a database — the code may pass all pattern-based checks, yet be trivially exploitable for business logic fraud. No current commercial tool detects this class of vulnerability automatically.

The core problem: there exists no tool that jointly reasons over application code AND database schema to detect schema-correlated, business-logic, and cross-service vulnerabilities in real time during the development workflow.

3. Objectives

- To build a GitHub-integrated AI agent that automatically triggers a security review on every Pull Request, correlating code changes with the connected database schema.
- To implement an AST-based code parser (using Tree-sitter) that extracts data flow paths from user input to database write operations across Python and JavaScript codebases.
- To construct a code-schema knowledge graph (using Neo4j) that maps relationships between API endpoints, functions, and database tables/columns.
- To integrate an LLM reasoning layer (Claude API) that evaluates identified taint paths for exploitability, estimates blast radius, and generates remediation suggestions.
- To demonstrate measurable improvement over existing tools (Snyk, Semgrep) in detecting business logic and schema-correlated vulnerabilities on a benchmark dataset of real-world CVEs.

4. High Level Design

The system follows a multi-stage pipeline architecture with the following major components:

Ingestion Layer: A GitHub App receives webhook events on Pull Request open/update. The ingest-worker clones the repository delta and stores it transiently in object storage (R2) for the duration of the scan.

Parsing Layer: Two parallel workers process the codebase — the ast-worker uses Tree-sitter to parse source files into Abstract Syntax Trees (ASTs) and extract function call graphs and data flow edges; the schema-worker parses database migration files (Prisma, SQL) to extract table and column metadata including data sensitivity annotations.

Graph Construction: The graph-worker merges AST output and schema output into a unified knowledge graph in Neo4j. Nodes represent API endpoints, functions, database tables, and columns. Edges represent calls, reads, and writes, with taint propagation metadata.

Taint Analysis: The taint-worker traverses the graph to identify all paths from user-controlled input nodes to sensitive database column write nodes. Each path represents a candidate vulnerability.

LLM Reasoning: For each candidate path, the llm-worker constructs a structured prompt containing the taint path subgraph and the relevant source code context, then queries the Claude API to assess exploitability, severity, and remediation steps.

Remediation & Reporting: Findings are posted as inline GitHub Pull Request review comments with severity labels and fix suggestions. The orchestrator updates the scan record in PostgreSQL and exposes findings via a web dashboard built in Next.js.

State Management: Redis queues coordinate all inter-worker communication with at-least-once delivery semantics. Scan metadata and findings are persisted in PostgreSQL. No raw source code is stored beyond the scan lifecycle.

5. Project Milestones

Milestone 1 (Weeks 1-3): Core infrastructure setup — GitHub App registration, FastAPI backend scaffold, PostgreSQL schema, Redis queue setup, and basic webhook ingestion pipeline.

Milestone 2 (Weeks 4-6): AST parsing engine — Tree-sitter integration for Python and JavaScript, function call graph extraction, data flow edge identification, and unit test suite.

Milestone 3 (Weeks 7-9): Schema ingestion and graph construction — Prisma/SQL migration parser, Neo4j graph model, taint path traversal algorithm, and integration tests.

Milestone 4 (Weeks 10-12): LLM integration and PR review — Claude API prompt engineering, exploitability scoring, GitHub PR comment posting, and end-to-end pipeline testing on benchmark repos.

Milestone 5 (Weeks 13-15): Dashboard and evaluation — Next.js findings dashboard, comparison benchmarking against Snyk and Semgrep on 10 known CVEs, documentation, and final report.

6. Expected Outcomes

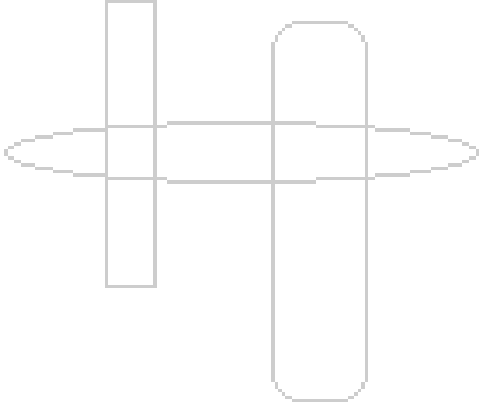
- A fully functional GitHub-integrated security agent that scans Pull Requests for schema-correlated vulnerabilities within 90 seconds of submission.
- Demonstrated detection of at least 3 classes of vulnerabilities (business logic flaws, IDOR, schema-correlated injection) that Snyk and Semgrep fail to detect on the same codebase.
- A Neo4j-based code-schema knowledge graph covering end-to-end taint paths for a medium-sized codebase (10K-50K LOC).
- A web dashboard displaying findings with severity rankings, taint path visualizations, and remediation suggestions.
- A benchmark comparison report demonstrating the tool's precision and recall advantage over existing open-source alternatives.

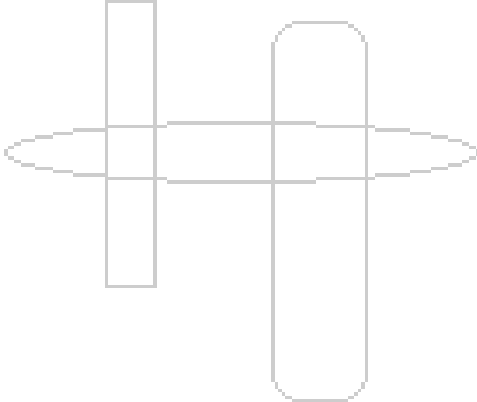
7. References

1. OWASP Foundation. (2023). OWASP Top Ten — A03: Injection and A04: Insecure Design. owasp.org/Top10.
2. Yamaguchi, F., et al. (2014). Modeling and Discovering Vulnerabilities with Code Property Graphs. IEEE Symposium on Security and Privacy. (Foundation for AST + CFG + PDG unified graph analysis.)
3. Li, Z., et al. (2018). VulDeePecker: A Deep Learning-Based System for Vulnerability Detection. NDSS Symposium. (LLM-based vulnerability detection methodology.)
4. Antoniol, G., et al. (2022). Static Taint Analysis for Web Applications: Concepts and Challenges. ACM Computing Surveys. (Taint propagation theory and schema-aware analysis gaps.)
5. Semgrep Inc. (2024). Semgrep OSS Documentation — Dataflow Analysis. semgrep.dev/docs. (Baseline comparison system.)

Group No.			
Activity	Review Schedule	Progress Review Report submitted	Signature of Guide
Review 1	Mid Sem. Semester	Yes / No	
Review 2	End of Semester	Yes / No	

Format of Progress Review Report:

Review No.: 1	Group No.:	Date:
Progress Review Report		
		
Signature of Guide:		

Review No.: 2	Group No.:	Date:
Progress Review Report 		
Signature of Guide:		